

C LLM Chat Prompt for Real-Robot Experiments

You are a helpful robot assistant.

To give you some knowledge of your environment, you are provided with a set of tags. Each tag describes something that's likely to be found in the environment. You can use these tags to help you with assisting the user.

Note that the tags are not perfect. Some tags may be incorrect and the tags do not cover everything in the environment. Therefore, try to pick the tag which you are most confident in.

You can query for the regions in the environment where a tag is localized and also get the confidence level for each region.

When making a statement about a tag, do not say that the tag is definitely in the environment, instead reference the confidence level.

If there are no tags directly related to the user's request, suggest tags which may be related to the user's request and ask the user if they would like to know more about the suggested tags.

List of tags in the format '[id] - [tag]'

0 - <tag 0>

1 - <tag 1>

⋮

N - <tag N>

D Details of the Evaluation Implementation

D.1 Grid Graph Construction

Computing the P2E and E2P requires solving for the shortest paths within a scene, which we compute approximately using a 3D grid graph that spans the scene without collision with the scene geometry. Note that the grid graph connects nodes without consideration of traversability. For example, the free space above a table would be connected but would not be traversable by a wheeled robot. An overview of the steps to construct a grid graph is given in Algorithm 1.

Algorithm 1 Scene Grid Graph Generation

```
1: Input: Scene Mesh  $\mathcal{M}$ 
2: Output: Set of nodes  $\mathcal{N}$  and edges  $\mathcal{E}$ 
3: Determine the min and max bounds of  $\mathcal{M}$ 
4: Generate a set of nodes  $\mathcal{N}$  following a grid spanning these bounds
5: for each node  $n \in \mathcal{N}$  do
6:   if InsideScene( $n, \mathcal{M}$ ) is false then
7:     Remove  $n$  from  $\mathcal{N}$ 
8:   end if
9: end for
10: Initialize empty set of edges  $\mathcal{E}$ 
11: for each node  $n \in \mathcal{N}$  do
12:   for each immediate neighbor  $n'$  of  $n$  in the grid do
13:     if  $(n, n') \notin \mathcal{E}$  and CollisionFree( $n, n', \mathcal{M}$ ) is true then
14:       Add edge  $(n, n')$  to  $\mathcal{E}$ 
15:     end if
16:   end for
17: end for
18: Return  $\mathcal{N}, \mathcal{E}$ 
```

The grid graphs are generated at a grid resolution of 0.5 m. We check if a point is inside the scene by checking its nearest neighboring vertices. First, if the mean distance to these vertices is greater than a threshold of 2 m we consider the point outside the scene. Second, we use the normals of each neighboring vertex to check if the point is locally inside the mesh at that vertex by computing the dot product of the vertex-to-point vector and the normal. We average the dot products across all neighboring vertices and consider the point outside the mesh if the average exceeds a threshold of 0. To evaluate if two points are collision-free, we cast a ray from one to the other and check for collisions against the scene mesh.

D.2 Assigning Grid Graph Nodes

The assignment of grid graph nodes to instance proposals is given in Algorithm 2. For assigning nodes to labeled entity instances, the algorithm differs depending on whether the entity is an object or a region, as outlined in Algorithms 3 and 4 respectively.

The node assignment algorithms differ in the use of inflation to assign additional nodes. In the case of instance proposals, inflation is only used when the proposal does not contain any nodes. Here we set the inflation amount δ such that the extent of the inflated proposal is at least larger than the space between nodes in the grid graph. For object labels, inflation is always used to assign additional nodes. In this case, we set δ to capture nodes 1 m away from the label bounding box, following the success criteria of the Habitat ObjectNav Challenge [2]. Lastly, no inflation is used for region labels as they are large enough to contain at least some nodes.

If no nodes can be assigned to an instance proposal or labeled entity, then the P2E or E2P cannot be computed. In these cases, we ignore that instance proposal or labeled entity when later computing the precision and recall metrics.

Algorithm 2 Assign Grid Graph Nodes to an Instance Proposal

```
1: Input: Set of nodes  $\mathcal{N}$ , Instance proposal bounding box  $P$ , Scene mesh  $\mathcal{M}$ , Inflation amount  $\delta$ 
2: Output: Set of nodes  $\mathcal{N}_{\text{result}}$ 
3: Initialize  $\mathcal{N}_{\text{result}} \leftarrow \emptyset$ 
4: Find nodes  $\mathcal{N}_{\text{in}} \subseteq \mathcal{N}$  that are contained within  $P$ 
5: if  $\mathcal{N}_{\text{in}} \neq \emptyset$  then
6:   return  $\mathcal{N}_{\text{in}}$ 
7: end if
8: Inflate  $P$  by a fixed amount  $\delta$  to obtain a new bounding box  $P'$ 
9: Find nodes  $\mathcal{N}_{\text{in}} \subseteq \mathcal{N}$  that are contained within  $P'$ 
10: for each node  $n \in \mathcal{N}_{\text{in}}$  do
11:   Find the nearest point  $p \in P$  to  $n$ 
12:   if  $\text{CollisionFree}(n, p, \mathcal{M})$  is true then
13:     Add  $n$  to  $\mathcal{N}_{\text{result}}$ 
14:   end if
15: end for
16: return  $\mathcal{N}_{\text{result}}$ 
```

Algorithm 3 Assign Grid Graph Nodes to a Labeled Object Instance

```
1: Input: Set of nodes  $\mathcal{N}$ , Object label bounding box  $O$ , Scene mesh  $\mathcal{M}$ , Inflation amount  $\delta$ 
2: Output: Set of nodes  $\mathcal{N}_{\text{result}}$ 
3: Find nodes  $\mathcal{N}_{\text{in}} \subseteq \mathcal{N}$  that are contained within  $O$ 
4:  $\mathcal{N}_{\text{result}} \leftarrow \mathcal{N}_{\text{in}}$ 
5: Inflate  $O$  by a fixed amount  $\delta$  to obtain a new bounding box  $O'$ 
6: Find additional nodes  $\mathcal{N}'_{\text{in}} \subseteq \mathcal{N}$  that are contained within  $O' \setminus O$ 
7: for each node  $n \in \mathcal{N}'_{\text{in}}$  do
8:   Find the nearest point  $p \in O$  to  $n$ 
9:   if  $\text{CollisionFree}(n, p, \mathcal{M})$  is true then
10:    Add  $n$  to  $\mathcal{N}_{\text{result}}$ 
11:   end if
12: end for
13: return  $\mathcal{N}_{\text{result}}$ 
```

Algorithm 4 Assign Grid Graph Nodes to a Labeled Region Instance

```
1: Input: Set of nodes  $\mathcal{N}$ , Region label bounding box  $R$ , Scene mesh  $\mathcal{M}$ 
2: Output: Set of nodes  $\mathcal{N}_{\text{result}}$ 
3: Find nodes  $\mathcal{N}_{\text{in}} \subseteq \mathcal{N}$  that are contained within  $R$ 
4:  $\mathcal{N}_{\text{result}} \leftarrow \mathcal{N}_{\text{in}}$ 
5: return  $\mathcal{N}_{\text{result}}$ 
```
